

Codificação Para Front-end

18

2026

SENAI - CRTI

Manipulação do DOM

Transforme dados estáticos em uma página viva — aprenda a renderizar produtos dinamicamente no seu e-commerce usando JavaScript e o Document Object Model.

DOM - Javascript e HTML



O Problema: Dados Existem, mas Não Aparecem

O que você tem

```
const produtos = [  
  {  
    id: 1,  
    nome: "Smartphone",  
    preco: 7500,  
    categoria: "Eletrônicos",  
    imagem: "img/iphone.jpg"  
  },  
  {  
    .  
    .  
    .  
  }  
]
```

O que o usuário vê

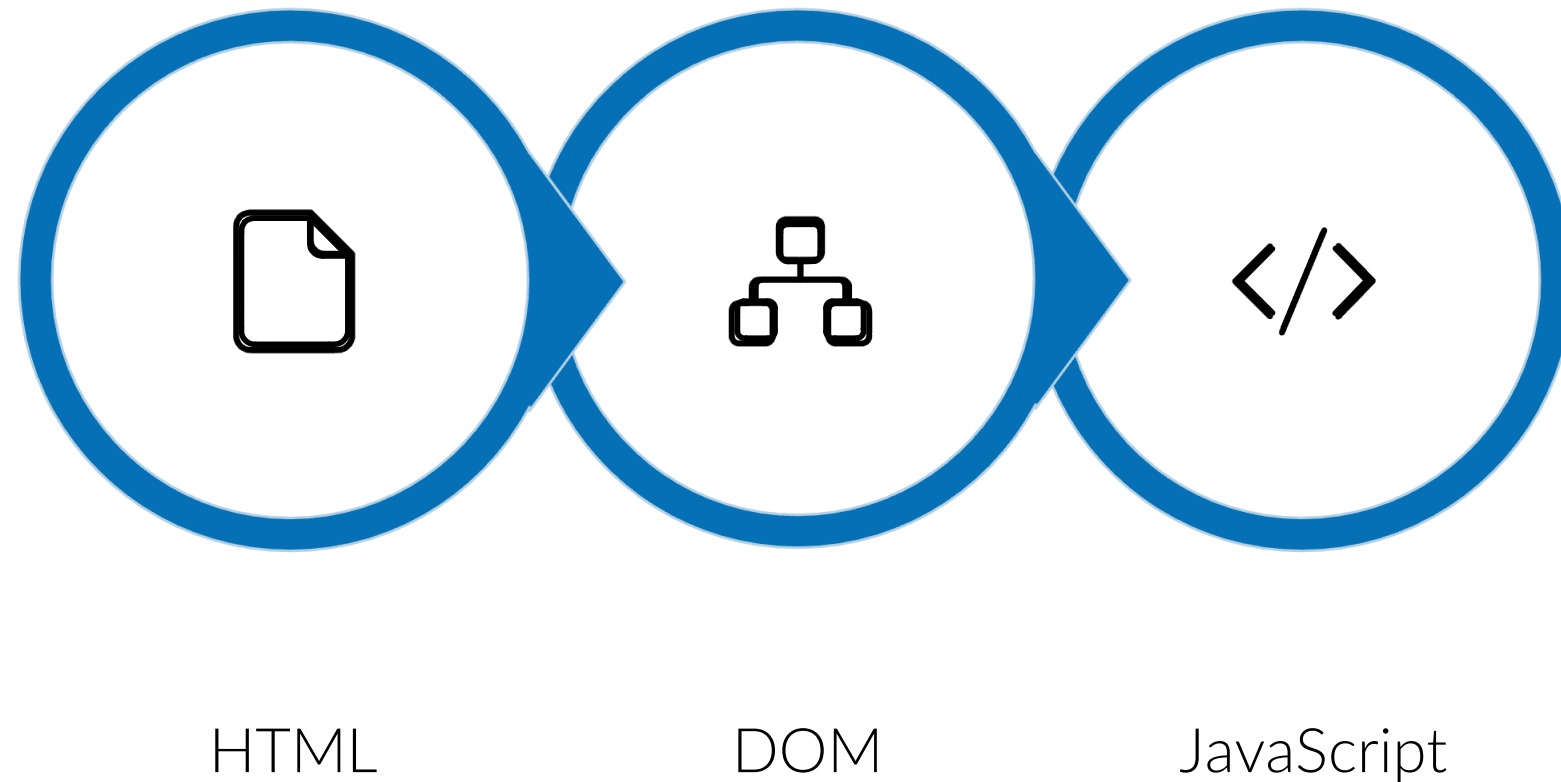
Uma página **completamente vazia** na seção de produtos. Os dados estão no JavaScript, mas o navegador não sabe como exibi-los automaticamente.

- ⚠ O HTML estático não conversa com o JavaScript sozinho — precisamos de uma ponte. Essa ponte é o **DOM**.



O que é o DOM?

DOM significa **Document Object Model** — é a representação em árvore de todos os elementos da sua página HTML. Quando o navegador carrega o HTML, ele cria essa estrutura na memória, e o JavaScript pode lê-la e modificá-la em tempo real.



Pense no DOM como um mapa interativo da sua página. O JavaScript usa esse mapa para encontrar elementos, alterar textos, adicionar novos cards de produto ou remover itens — tudo sem recarregar a página.

A Árvore do DOM

Estrutura HTML

```
<html>
  <body>
    <h1>Loja</h1>
    <div id="product-list">
      <div class="card">
        <p>Camiseta</p>
      </div>
    </div>
  </body>
</html>
```

Como o DOM enxerga

document (raiz)

↳ **html**

↳ **body**

↳ **h1** → "Loja"

↳ **div#product-list**

↳ **div.card**

↳ **p** → "Camiseta"

Cada elemento HTML vira um **nó** na árvore. Nós têm relações de **pai**, **filho** e **irmão** — e o JavaScript pode navegar por essas relações para localizar e modificar qualquer parte da página.

Capacidades desta Aula



Selecionar Elementos

Usar `querySelector` e `getElementById` para localizar elementos na página.



Criar Elementos

Gerar novos elementos HTML via `createElement` e inseri-los com `appendChild`.



Alterar Conteúdo

Modificar textos e atributos HTML dinamicamente com `innerHTML` e `textContent`.



Remover Elementos

Excluir nós do DOM com `removeChild` e `remove()`.

Seletores: Encontrando Elementos

Para manipular qualquer elemento, primeiro você precisa **selecioná-lo**. O JavaScript oferece métodos poderosos para isso:

getElementById

```
const container =  
document.getElementById("pr  
oduct-list");  
// Seleciona: <div  
id="produtos">
```

Busca um elemento pelo seu **id**. Retorna um único elemento. É o método mais rápido quando você tem um ID único.

querySelector

```
const titulo =  
document.querySelector("h1"  
);  
const card =  
document.querySelector(".ca  
rd");  
const btn =  
document.querySelector("#bt  
n-comprar");  
// Usa seletores CSS!  
Aceita qualquer seletor CSS e retorna  
o primeiro elemento encontrado. Mais  
flexível.
```

querySelectorAll

```
const cards =  
document.querySelectorAll("  
.card");  
// Retorna todos os  
elementos .card
```

Retorna **todos** os elementos que correspondem ao seletor, em uma **NodeList** que pode ser percorrida com **forEach**.

innerHTML vs textContent

Uma vez que você tem um elemento selecionado, pode alterar o seu conteúdo. As duas propriedades mais usadas são `innerHTML` e `textContent` — e é importante saber a diferença:

innerHTML

```
const card = document.querySelector(".card");

// Insere HTML de verdade
card.innerHTML = `
  
  <h3>${produto.nome}</h3>
  <p>R$ ${produto.preco}</p>
`;
```

Interpreta e renderiza **tags HTML**. Use quando precisar inserir estrutura completa. ⚠ Cuidado com dados vindos do usuário.

textContent

```
const titulo = document.querySelector("h1");

// Insere apenas texto puro
titulo.textContent = "Bem-vindo à nossa loja!";

// <b>negrito</b> vira texto literal
// NÃO interpreta HTML
```

Insere apenas **texto puro**, sem interpretar HTML. Mais seguro para exibir dados do usuário. Ideal para títulos e parágrafos simples.

Criando Elementos Dinamicamente

Em vez de escrever HTML estático, o JavaScript pode **criar elementos do zero** e adicioná-los à página. Esse é o coração da renderização dinâmica:

```
// 1. Criar o elemento
const card = document.createElement("div");

// 2. Adicionar classe
const card = document.createElement("div");
card.classList.add("product-card");
```

Removendo Elementos

Método moderno: `remove()`

```
// Seleciona e remove diretamente
const card = document.querySelector(".card-
produto");
card.remove();

// Com base em dado dinâmico
function removerProduto(id) {
  const el = document.getElementById("prod-" +
id);
  if (el) el.remove();
}
```

Forma mais simples e direta. Suportado por todos os navegadores modernos. **Recomendado.**

Método clássico: `removeChild`

```
// Precisa do elemento pai
const pai =
document.getElementById("produtos");
const filho = document.querySelector(".card");

pai.removeChild(filho);

// Alternativa: limpar tudo
pai.innerHTML = "";
```

Exige referência ao elemento pai. Útil para entender a relação pai-filho no DOM.

✅ Dica prática: use `pai.innerHTML = ""` para limpar o container de produtos antes de renderizar novamente — evita duplicatas!

Navegando na Árvore do DOM

O DOM é uma árvore, e o JavaScript permite **navegar** pelos nós sem precisar usar seletores CSS. Isso é especialmente útil quando você está dentro de um evento e precisa acessar elementos vizinhos.

Pai

```
el.parentElement
```

Sobe um nível — acessa o elemento que contém o atual.

Filhos

```
el.children  
el.firstChild  
el.lastChild
```

Acessa os elementos filhos diretos.

Irmãos

```
el.nextElementSibling  
el.previousElementSibling
```

Navega entre elementos no mesmo nível.

```
// Exemplo: a partir de um botão, acessar o card-pai e remover  
const btn = document.querySelector(".btn-remover");  
const cardPai = btn.parentElement; // sobe para o .card  
cardPai.remove(); // remove o card inteiro
```

Navegando na Árvore do DOM

```
lista.forEach(produto => {  
  const card =  
    document.createElement("div");  
  
  card.classList.add("product-card");  
  
  card.innerHTML = `  
      
    <h3>${produto.nome}</h3>  
    <p>R$ ${produto.preco}</p>  
  `;  
  
  container.appendChild(card);  
});
```

Manipulando Atributos e Classes

Atributos

```
const img = document.querySelector("img");

// Ler atributo
img.getAttribute("src"); // "camisa.jpg"

// Definir atributo
img.setAttribute("src", "nova-camisa.jpg");
img.setAttribute("alt", "Camiseta Atualizada");

// Verificar
img.hasAttribute("alt"); // true

// Remover
img.removeAttribute("alt");
```

```
card.classList.add("product-card");
```

Classes CSS

```
const card = document.querySelector(".card");

// Adicionar classe
card.classList.add("destaque");

// Remover classe
card.classList.remove("destaque");

// Alternar (toggle)
card.classList.toggle("ativo");
// Se não tem → adiciona
// Se tem → remove

// Verificar
card.classList.contains("ativo"); // true/false
```

```

```

Atividade Prática: Mãos no Código!

Agora é hora de aplicar tudo no projeto real. Vamos construir a renderização dinâmica dos produtos do nosso e-commerce, passo a passo.

01

Preparar o HTML

Adicionar o container `<div id="product-list">` na página.

02

Selecionar o container

Usar `getElementById` no JS para obter a referência.

03

Percorrer o array

Usar `forEach` no array de produtos da aula anterior.

04

Criar e inserir cards

Para cada produto, criar um `div.card` e inserir

05

Testar inserção e remoção

Adicionar um produto manualmente e removê-lo pelo console.

HTML Fixo vs. DOM Dinâmico

✗ Antes: HTML Fixo

Cada produto era escrito manualmente no HTML. Para adicionar um novo item era necessário editar o arquivo `.html`, duplicar o código e atualizar manualmente nome, preço e imagem. Trabalhoso e propenso a erros.

```
<div class="card">
  <h3>Camiseta</h3>
  <p>R$ 49,90</p>
</div>
<!-- repetir para cada produto... -->
```

✓ Agora: DOM Dinâmico

O JavaScript lê o array de produtos e gera o HTML automaticamente. Para adicionar um produto, basta adicionar um objeto ao array — a página se atualiza sozinha. Escalável, limpo e profissional.

```
renderizarProdutos(produtos);
```

Destructing

```
const { nome, preco } = produtos[0];  
console.log(`Produto: ${nome} - R$ ${preco}`);
```

Estamos pegando diretamente o **nome** e o **preço** do primeiro produto da lista sem precisar escrever `produtos[0].nome` toda hora.

Filtrar

```
function filtrarPorCategoria(categoria) {  
  return produtos.filter(  
    produto => produto.categoria === categoria  
  );  
}
```

Percorremos todos os produtos e guardamos apenas os que pertencem à categoria Eletrônicos.

Listar

```
function listarProdutos(lista) {  
  lista.forEach(produto => {  
    console.log(  
      `Produto: ${produto.nome} - R$  
      ${produto.preco}`  
    );  
  });  
}
```

O JavaScript pega cada item do array e executa esse bloco de código para cada produto.

JSON

```
const produtosJSON = JSON.stringify(produtos);  
const produtosConvertidos =  
  JSON.parse(produtosJSON);
```

Spread

```
const novosProdutos = [  
  ...produtos,  
  {  
    id: 4,  
    nome: "Notebook",  
    preco: 3500,  
    categoria: "Eletrônicos",  
    imagem: "https://via.placeholder.com/150"  
  }  
];
```

Aqui copiamos todos os produtos antigos e adicionamos mais um novo produto no final.

Boas Práticas de DOM

→ Cache suas referências

Selecione elementos uma vez e guarde em variável. Evite chamar `getElementById` repetidamente dentro de loops — isso é lento.

→ Use DocumentFragment para listas grandes

Em vez de fazer `appendChild` em cada iteração do loop, monte tudo em um `DocumentFragment` e insira uma vez só — melhora a performance.

→ Comente o código

Adicione comentários explicando *por que* você fez algo, não apenas o *que* o código faz. Isso vale ouro na hora de revisitar o projeto.



